

MIT 212 — Programming Techniques

Master of Information Technology · Sabaragamuwa University of Sri Lanka

Question Bank

ප්‍රශ්න බැංකුව — 5, 6 සහ 7 සති

Weeks 5–7: 2D Arrays · Structures · Git Branching & Conflicts · Pointers & Memory

1. Multiple-Choice Questions (20) with Answer Key
2. True/False Questions with Corrections (10)
3. Definitions & Short-Answer Questions (10)
4. Comparison Questions (5)
5. Code-Output Questions (8)
6. Error-Identification & Debugging Questions (6)
7. Algorithm / Pseudocode Questions (3)
8. Flowchart Questions (2)
9. Short Programming Questions (5)
10. Substantial Practical Problem with Model Solution & Marking Scheme

Bilingual Edition — English / සිංහල

11. Multiple-Choice Questions (20) බහුවරණ ප්‍රශ්න (20)

Choose the ONE best answer. Answers are given at the end of this section.

හොඳම පිළිතුර එකක් තෝරන්න. පිළිතුරු මෙම කොටස අවසානයේ ලබා දී ඇත.

Q1. Given matrix = [[1,2,3],[4,5,6],[7,8,9]], what does matrix[1][2] return?

ඉහත matrix හි matrix[1][2] ලබා දෙන්නේ කුමක්ද?

- A) 2
- B) 4
- C) 6
- D) 8

Q2. What does len(matrix[0]) give for a 2D array?

ද්විමාන අරාවක len(matrix[0]) ලබා දෙන්නේ?

- A) Number of rows
- B) Number of columns
- C) Total elements
- D) The first value

Q3. Executing print(matrix[3][0]) on a 3x3 matrix causes:

3x3 matrix එකක matrix[3][0] මුද්‍රණය කිරීමෙන් සිදුවන්නේ:

- A) A logical error
- B) IndexError at runtime
- C) A syntax error
- D) It prints 0

Q4. How many loops are needed to traverse every cell of a 2D array?

ද්විමාන අරාවක සෑම කොටුවක්ම ගමන් කිරීමට අවශ්‍ය ලූප ගණන:

- A) One
- B) Two (nested)
- C) Three
- D) None — use print(matrix)

Q5. When should you prefer an index-based loop (range(len(...)))?

දර්ශක පාදක ලූපයක් තෝරාගත යුත්තේ කවදාද?

- A) When you need to know WHERE you are (coordinates)
- B) When you only need values
- C) Only for 3D arrays
- D) Never — value loops are always better

Q6. In print(value, end=" "), the purpose of end=" " is to:

end=" " හි අරමුණ:

- A) Add a newline
- B) Keep values on the same line
- C) End the loop
- D) Skip empty values

Q7. A structure differs from an array because a structure:

ව්‍යුහයක් අරාවකින් වෙනස් වන්නේ:

- A) Holds only numbers
- B) Groups mixed types in one container
- C) Cannot be looped
- D) Is always faster

Q8. What does the @dataclass decorator do?

@dataclass සැරසිලිකාරකය කරන්නේ:

- A) Makes the class immutable
- B) Generates __init__ automatically
- C) Connects to a database
- D) Sorts the fields

Q9. s3 = Student(25, "Amal", 3.8) for fields (name, age, gpa) is wrong because:

එය වැරදි වන්නේ:

- A) Too many values
- B) Values are not in field order — age gets "Amal"
- C) gpa must be int
- D) Student needs keywords

Q10. students[2].gpa means:

students[2].gpa හි තේරුම:

- A) Field 2 of gpa
- B) Index into the list first, then access the field
- C) The gpa of every student
- D) A syntax error

Q11. Which command creates a branch AND switches to it?

ශාඛාවක් සාදා එයටම මාරු වන විධානය:

- A) git branch feature
- B) git merge feature
- C) git checkout -b feature
- D) git switch --create

Q12. In git branch output, the asterisk (*) marks:

git branch ප්‍රතිදානයේ * සලකුණ:

- A) The oldest branch
- B) The branch you are currently on
- C) A merged branch
- D) A remote branch

Q13. A fast-forward merge happens when:

Fast-forward merge සිදුවන්නේ:

- A) Both branches have new commits
- B) main had no new commits since branching
- C) There is a conflict
- D) The branch was deleted

Q14. git branch -d feature will refuse to run if:

git branch -d ප්‍රතික්ෂේප වන්නේ:

- A) The branch is merged
- B) The branch has NOT been merged
- C) You are on main
- D) The repo is new

Q15. A merge conflict occurs when:

එකාබද්ධ ගැටුමක් ඇතිවන්නේ:

- A) Two branches change the same line of the same file
- B) Two branches exist at once
- C) You forget to push
- D) A file is renamed

Q16. In conflict markers, the code between <<<<<< HEAD and ===== is:

<<<<<< HEAD සහ ===== අතර කේතය:

- A) The incoming branch's version
- B) YOUR version (current branch)
- C) Git's suggestion
- D) Deleted code

Q17. What does id(x) return in Python?

Python හි id(x) ලබා දෙන්නේ:

- A) The value of x
- B) The memory address where x lives
- C) The type of x

D) The size of x

Q18. After `x = [42]; ptr = x; ptr[0] = 99`, what is `x[0]`?

`x = [42]; ptr = x; ptr[0] = 99` ට අසු `x[0]`:

- A) 42
- B) 99
- C) IndexError
- D) None

Q19. After `b = 10; ref = b; ref = 50`, what is `b`?

`b = 10; ref = b; ref = 50` ට අසු `b`:

- A) 50
- B) 10
- C) 60
- D) Error

Q20. Python's equivalent of C's `*(ptr+i)` is:

C හි `*(ptr+i)` ට Python සමානයි:

- A) `ptr.i`
- B) `arr[i]`
- C) `arr{i}`
- D) `id(arr)+i`

MCQ Answer Key බහුවරණ පිළිතුරු

Q	Ans	Why / හේතුව
1	C	Row 1 is [4,5,6]; column 2 of that row is 6.
2	B	<code>len(matrix) = rows</code> ; <code>len(matrix[0]) = length of the first row = columns</code> .
3	B	Row 3 does not exist (valid rows 0–2) → runtime <code>IndexError</code> .
4	B	One loop over rows, an inner loop over the values in each row.
5	A	Index loops give you the exact <code>[r][c]</code> coordinates for search/modify tasks.
6	B	It replaces the default newline with a space.
7	B	Array = same type repeated; structure = mixed-type fields under one name.
8	B	It marks the class as a data container and auto-generates <code>__init__</code> .
9	B	Values must follow the declared field order.
10	B	Index FIRST to get the object, then dot notation for the field.
11	C	<code>git checkout -b = git branch + git checkout</code> in one step.
12	B	Every commit you make goes to the starred branch.
13	B	Git simply moves the main pointer forward — no merge commit needed.
14	B	<code>-d</code> is the safe delete; use <code>-D</code> only when you accept losing the commits.
15	A	Git cannot decide which version to keep, so it pauses and asks you.
16	B	HEAD = current branch = your side; below ===== is theirs.
17	B	<code>id()</code> is Python's address-of — equivalent in concept to <code>&x</code> in C.
18	B	Lists are mutable — <code>ptr</code> and <code>x</code> reference the SAME object.
19	B	Integers are immutable — <code>ref = 50</code> binds <code>ref</code> to a NEW object; <code>b</code> is untouched.
20	B	Same concept, different syntax; in C, <code>ptr+1</code> advances <code>sizeof(int) = 4</code> bytes.

12. True/False Questions with Corrections (10) සත්‍ය/අසත්‍ය ප්‍රශ්න සහ නිවැරදි කිරීම් (10)

1. In `matrix[row][col]`, the column index comes first.

FALSE — Correction: The ROW index always comes first: `matrix[row][col]`.

පේළි දර්ශකය සැමවිටම පළමුවෙනි.

2. `len(matrix)` returns the number of rows of a 2D array.

TRUE

සත්‍යයි — `len(matrix)` පේළි ගණන ලබා දෙයි.

3. A structure can only store fields of the same data type.

FALSE — Correction: A structure groups fields of DIFFERENT types (str, int, float); an array repeats the same type.

ව්‍යුහයක් විවිධ වර්ග කාණ්ඩ කරයි.

4. `@dataclass` automatically generates the `__init__` method.

TRUE

සත්‍යයි — `__init__` ස්වයංක්‍රීයව සෑදේ.

5. It is best practice to commit your daily work directly on main.

FALSE — Correction: Never work directly on main — always create a feature branch first (git checkout -b).

සැමවිටම පළමුව ශාඛාවක් සාදන්න.

6. `git branch -D` deletes a branch even if its commits are unmerged.

TRUE

සත්‍යයි — එබැවින් ප්‍රවේශමෙන් භාවිතා කරන්න; commits අහිමි වේ.

7. A 3-way merge happens when main has no new commits.

FALSE — Correction: That is a FAST-FORWARD. A 3-way merge happens when BOTH branches have new commits.

එය fast-forward වේ; 3-way merge ශාඛා දෙකේම නව commits ඇති විටය.

8. Conflict markers may remain in the file after resolving, as comments.

FALSE — Correction: NO markers may remain — Git/Python treat leftover <<< === >>> as syntax errors.

සලකුණු කිසිවක් ඉතිරි විය නොහැක.

9. In Python, `b = a` on a list creates an independent copy of the list.

FALSE — Correction: It creates a REFERENCE to the same object; use `a.copy()` or `copy.deepcopy(a)` for independence.

එය එකම වස්තුවට යොමුවකි; පිටපතකට `.copy()/deepcopy()`.

10. `a == b` can be True while `a is b` is False.

TRUE

සත්‍යයි — `==` අගය ද `is` අනන්‍යතාව ද සංසන්දනය කරයි; `[1,2,3]` දෙකක් සමාන නමුත් වෙනස් වස්තුය.

13. Definitions & Short-Answer Questions (10) අර්ථ දැක්වීම් සහ කෙටි පිළිතුරු ප්‍රශ්න (10)

Q1. Define traversal and state why we traverse a 2D array.

ගමන් කිරීම අර්ථ දක්වා අප ද්විමාන අරාවක් ගමන් කරන්නේ ඇයිද යන්න සඳහන් කරන්න.

Model answer: Traversal is visiting every element in a data structure one by one, in order, without skipping any. We traverse to search, sum, compare, print, or modify every cell — any operation that must touch each value. A 2D array needs two nested loops.

ගමන් කිරීම යනු සෑම මූලද්‍රව්‍යයක්ම පිළිවෙලින්, මඟ නොහැර බැලීමයි. සෙවීම, එකතු කිරීම, සංසන්දනය, මුද්‍රණය හෝ වෙනස් කිරීම සඳහා ගමන් කරමු.

Q2. What is a structure, and how does it differ from an array?

ව්‍යුහයක් යනු කුමක්ද? එය අරාවකින් වෙනස් වන්නේ කෙසේද?

Model answer: A structure is a custom data type that groups related fields of different types under one name, like a labelled form. An array repeats the same type; a structure mixes types in one container.

ව්‍යුහයක් යනු විවිධ වර්ගවල ක්ෂේත්‍ර එක් නමක් යටතේ කාණ්ඩ කරන අභිරුචි දත්ත වර්ගයකි. අරාව එකම වර්ගය පුනරාවර්තනය කරයි.

Q3. Explain what a Git branch is and give two benefits of branching.

Git ශාඛාවක් පැහැදිලි කර ප්‍රතිලාභ දෙකක් දෙන්න.

Model answer: A branch is an independent copy of the code timeline; changes do not affect main until merged. Benefits: (1) safe experimentation — main stays clean; (2) team independence — several developers work on different features simultaneously; (3) merge only when tested.

ශාඛාවක් යනු කේත කාලරේඛාවේ ස්වාධීන පිටපතකි. ප්‍රතිලාභ: ආරක්ෂිත අත්හදාබැලීම්, කණ්ඩායම් ස්වාධීනත්වය.

Q4. What is a merge conflict and when exactly does it occur?

එකාබද්ධ ගැටුමක් යනු කුමක්ද? එය ඇතිවන්නේ කවදාද?

Model answer: A conflict occurs when two branches change the SAME line in the SAME file. Git cannot decide which version to keep, writes conflict markers into the file, and pauses the merge for a human decision.

ශාඛා දෙකක් එකම ගොනුවේ එකම පේළිය වෙනස් කළ විට; Git තීරණය කළ නොහැකි නිසා නවතී.

Q5. List the five steps for resolving a merge conflict.

ගැටුමක් විසඳීමේ පියවර පහ ලැයිස්තුගත කරන්න.

Model answer: 1) git status (shows both modified); 2) open the file and find <<<<<< markers; 3) choose yours, theirs, or combine both; 4) save and remove ALL markers; 5) git add . and git commit.

1) git status 2) ගොනුව විවෘත කර සලකුණු සොයන්න 3) අනුවාදය තෝරන්න 4) සියලු සලකුණු ඉවත් කරන්න 5) add සහ commit.

Q6. Define value and address, and name the operator/function for the address in Python and in C.

අගය සහ ලිපිනය අර්ථ දක්වා Python සහ C හි ලිපින කාරකය නම් කරන්න.

Model answer: The value is what a variable stores (25, "Nimal"); the address is where it lives in RAM. Python: id(x); C: &x. Both return the memory location.

අගය = ගබඩා කරන දේ; ලිපිනය = RAM හි ස්ථානය. Python: id(x); C: &x.

Q7. What is a pointer? How does Python handle pointers compared with C?

දර්ශකයක් යනු කුමක්ද? Python එය C ට සාපේක්ෂව හසුරුවන්නේ කෙසේද?

Model answer: A pointer is a variable that stores the memory address of another variable. In C, pointers are explicit (declared with *, address taken with &). In Python, every variable is already a reference under the hood — assignment shares the object automatically.

දර්ශකය ලිපිනයක් ගබඩා කරයි. C හි පැහැදිලිය (* සහ &); Python හි සෑම විචල්‍යයක්ම දැනටමත් යොමුවකි.

Q8. Explain dereferencing with the C example *ptr.

*ptr උදාහරණයෙන් dereference පැහැදිලි කරන්න.

Model answer: Dereferencing means following the pointer: go to the address it holds and read (or change) the value stored there. If ptr holds 1000 and address 1000 stores 42, *ptr reads 42; *ptr = 99 changes x itself.

Dereference = ලිපිනයට ගොස් අගය කියවීම/වෙනස් කිරීම. *ptr = 99 මගින් x ම වෙනස් වේ.

Q9. What is a pointer to pointer (**)? Give one real-world use.

දර්ශකයට දර්ශකය (**) යනු කුමක්ද? සැබෑ භාවිතයක් දෙන්න.

Model answer: It stores the address OF a pointer — two levels of indirection, like a signpost pointing to another signpost. Uses: 2D structures (int** rows), command-line arguments (char **argv ≡ sys.argv), linked lists/trees, letting a function modify what a pointer points to.

දර්ශකයක ලිපිනය ගබඩා කරයි — වක්‍ර මට්ටම් දෙකකි. භාවිත: ද්විමාන ව්‍යුහ, char **argv, linked lists.

Q10. Distinguish mutable from immutable types and state the 'golden rule' of copying.

විකාර්ය/අවිකාර්ය වර්ග වෙන්කර පිටපත් කිරීමේ රන් රීතිය සඳහන් කරන්න.

Model answer: Mutable (list, dict, object) can change in place, so references share changes; immutable (int, str, tuple) cannot — reassignment creates a new object. Golden rule: to share, assign directly; for independence, use .copy() (flat) or copy.deepcopy() (nested).

විකාර්ය ස්ථානීයව වෙනස් වේ; අවිකාර්ය නව වස්තුවක් සාදයි. රන් රීතිය: බෙදාගැනීමට කෙලින්ම පවරන්න; ස්වාධීනත්වයට .copy()/deepcopy().

14. Comparison Questions (5) සංසන්දන ප්‍රශ්න (5)

Q1. Compare a value-based loop with an index-based loop for 2D array traversal. When is each appropriate?

අගය පාදක සහ දර්ශක පාදක ලූප සංසන්දනය කරන්න.

Model answer: Value-based (for value in row) is simpler and appropriate when you only need the values — printing, summing. Index-based (for r in range(len(m)): for c in range(len(m[r]))) gives the exact coordinates [r][c] — required for searching (recording where a value is), modifying specific cells (m[r][c] = new), and position-dependent comparisons.

Q2. Compare git branch -d with git branch -D.

git branch -d සහ -D සංසන්දනය කරන්න.

Model answer: -d is the safe delete: Git checks the branch is already merged and refuses if work would be lost. -D force-deletes even if unmerged — those commits are permanently lost. Best practice: merge first, then -d to keep the repo clean.

Q3. Compare a fast-forward merge with a 3-way merge.

Fast-forward සහ 3-way merge සංසන්දනය කරන්න.

Model answer: Fast-forward: main gained no new commits since branching, so Git simply moves main's pointer to the branch tip — no new commit. 3-way: both branches gained commits, so Git combines both histories and creates a dedicated merge commit ('recursive' strategy). If both changed the same line, the 3-way merge pauses with a conflict.

Q4. Compare how Python and C expose memory addresses and dereference them.

Python සහ C මතක ලිපින හඳුන්වන ආකාරය සංසන්දනය කරන්න.

Model answer: Address-of: Python id(x); C &x. Pointer variable: Python ptr = x (implicit reference); C int *ptr = &x (explicit). Dereference: Python normally automatic (ptr[0], obj.field) — the advanced form is ctypes.cast(id(x), ctypes.py_object).value; C uses *ptr. Struct access: Python always uses the dot; C uses . for the struct itself and -> through a pointer.

Q5. Compare shallow copy (.copy()) with deep copy (copy.deepcopy()).

.copy() සහ deepcopy() සංසන්දනය කරන්න.

Model answer: list.copy() copies only the outer list — inner (nested) lists are still shared references, so b[0][0] = 99 also changes a. copy.deepcopy() recursively copies every level, producing a fully independent structure. Use .copy() for flat lists, deepcopy() for nested lists or objects.

15. Code-Output Questions (8) කේත ප්‍රතිදාන ප්‍රශ්න (8)

Predict the exact output of each program before checking the answer.

පිළිතුර බලීමට පෙර සෑම වැඩසටහනකම නිවැරදි ප්‍රතිදානය පුරෝකථනය කරන්න.

Q1. What is the output?

ප්‍රතිදානය කුමක්ද?

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(matrix[1][2])
matrix[0][1] = 99
print(matrix[0])
```

Output: 6 then [1, 99, 3]

matrix[1][2] = row 1, col 2 = 6. Then row 0 becomes [1, 99, 3].

Q2. What is the output?

ප්‍රතිදානය කුමක්ද?

```
matrix = [[2, 4], [6, 8]]
total = 0
for r in range(len(matrix)):
    for c in range(len(matrix[r])):
        total += matrix[r][c]
print("Sum:", total)
```

Output: Sum: 20

2 + 4 + 6 + 8 = 20.

Q3. What is the output?

ප්‍රතිදානය කුමක්ද?

```
for row in [[1, 2], [3, 4]]:
    for v in row:
        print(v, end=" ")
    print()
```

Output: 1 2 (newline) 3 4

end=" " keeps values on one line; print() starts a new row.

Q4. What is the output?

ප්‍රතිදානය කුමක්ද?

```
from dataclasses import dataclass
@dataclass
class Student:
    name: str
    age: int
    gpa: float
s1 = Student("Amal", 25, 3.8)
s1.gpa = 3.9
print(s1)
```

Output: Student(name='Amal', age=25, gpa=3.9)

@dataclass also provides a readable printout; gpa was modified to 3.9.

Q5. What is the output?

ප්‍රතිදානය කුමක්ද?

```
students = [Student("Amal", 25, 3.8), Student("Nimal", 27, 3.5),
            Student("Kamala", 24, 3.9)]
best = students[0]
for s in students:
    if s.gpa > best.gpa:
        best = s
print(best.name)
```

Output: Kamala

Highest gpa is 3.9 — the classic find-the-maximum loop over an array of structs.

Q6. What is the output?

ප්‍රතිදානය කුමක්ද?

```
x = [42]
ptr = x
ptr[0] = 99
print(x[0], id(x) == id(ptr))
```

Output: 99 True

Lists are mutable — one shared object; both names see 99, ids are equal.

Q7. What is the output?

ප්‍රතිදානය කුමක්ද?

```
b = 10
ref = b
ref = 50
print(b, ref)
```

Output: 10 50

Integers are immutable — ref = 50 rebinds ref to a new object; b is unchanged.

Q8. What is the output?

ප්‍රතිදානය කුමක්ද?

```
import copy
a = [[1, 2], [3, 4]]
b = a.copy()
c = copy.deepcopy(a)
b[0][0] = 99
c[1][1] = 77
print(a)
```

Output: [[99, 2], [3, 4]]

.copy() is shallow — inner lists shared, so b[0][0]=99 hits a. deepcopy is independent, so c's change does not.

16. Error-Identification & Debugging Questions (6) දෝෂ හඳුනාගැනීම් සහ නිදෝෂීකරණ ප්‍රශ්න (6)

Each snippet contains at least one error. Name the error type (syntax / runtime / logical), identify it, and correct it.

සෑම කේත කැබලිලකම අවම වශයෙන් එක් දෝෂයක් ඇත. දෝෂ වර්ගය නම් කර, හඳුනාගෙන, නිවැරදි කරන්න.

Q1. Find and fix the error:

දෝෂය සොයා නිවැරදි කරන්න:

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]
print(matrix[3][0])
```

Answer: Runtime error (IndexError). A 3x3 matrix has rows 0–2; row 3 does not exist. Fix: use a valid index, e.g. matrix[2][0].

බාවන දෝෂය (IndexError) — පේළිය 3 නොපවතී; වලංගු පේළි 0–2.

Q2. Find and fix the error:

දෝෂය සොයා නිවැරදි කරන්න:

```
matrix = [[1,2],[3,4]]
total = 0
for row in matrix:
    total += row          # trying to add a list
print(total)
```

Answer: Runtime error (TypeError): adding a list to an int. The single loop yields ROWS, not values. Fix: add an inner loop — for v in row: total += v.

තනි ලූපය පේළි ලබා දෙයි — අගයන් නොවේ. අභ්‍යන්තර ලූපයක් එක් කරන්න.

Q3. Find and fix the error:

දෝෂය සොයා නිවැරදි කරන්න:

```
total = 1
for row in [[1,2],[3,4]]:
    for v in row:
        total += v
print(total) # expected 10
```

Answer: Logical error. The program runs but prints 11 because the accumulator starts at 1 instead of 0. Fix: total = 0.

තාර්කික දෝෂය — ආරම්භක අගය 1 නිසා 11 මුද්‍රණය වේ; total = 0 විය යුතුය.

Q4. Find and fix the error:

දෝෂය සොයා නිවැරදි කරන්න:

```
from dataclasses import dataclass
@dataclass
class Student:
    name: str
    age: int
    gpa: float
s = Student(25, "Amal", 3.8)
print(s.name.upper())
```

Answer: Runtime/logical error: values are in the wrong field order, so name = 25 (an int) and .upper() raises AttributeError. Fix: Student("Amal", 25, 3.8) — values must follow field order.

වැරදි ක්ෂේත්‍ර අනුපිළිවෙල — name = 25 වීමෙන් .upper() අසාර්ථක වේ.

Q5. Find and fix the error:

දෝෂය සොයා නිවැරදි කරන්න:

```
# after a merge, student.py still contains:
<<<<<< HEAD
gpa = 3.8
=====
```

```
gpa = 4.0
>>>>> branch-B
```

Answer: Syntax error when the file runs: conflict markers were left behind. Fix: choose one version (or combine), delete ALL <<< === >>> lines, then git add . and git commit.

ගැටුම් සලකුණු ඉතිරි වී ඇත — වාගේ රීති දෝෂ. එක් අනුවාදයක් තෝරා සලකුණු මකා add/commit කරන්න.

Q6. Find and fix the error:

දෝෂය සොයා නිවැරදි කරන්න:

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)      # programmer expected [1, 2, 3]
```

Answer: Logical error (reference vs copy): b = a shares the same list, so a becomes [1,2,3,4]. Fix: b = a.copy() for an independent flat copy.

b = a යොමුවකි — a ද වෙනස් වේ. ස්වාධීන පිටපතට b = a.copy().

17. Algorithm / Pseudocode Questions (3) ඇල්ගොරිතම / ව්‍යාජ කේත ප්‍රශ්න (3)

Q1. Write pseudocode to find the largest value in a 2D array and report its position [row][col].

ද්විමාන අරාවක විශාලතම අගය සහ එහි පිහිටීම [row][col] සොයන ව්‍යාජ කේතය ලියන්න.

```
SET max_val ← matrix[0][0], max_r ← 0, max_c ← 0
FOR r FROM 0 TO rows-1:
  FOR c FROM 0 TO cols-1:
    IF matrix[r][c] > max_val THEN
      max_val ← matrix[r][c]; max_r ← r; max_c ← c
OUTPUT max_val, max_r, max_c
```

An index-based traversal is required because the position must be recorded — a value-based loop cannot report coordinates.

Q2. Write the algorithm (numbered steps) for safely completing a feature using the GitHub Flow.

GitHub Flow භාවිතයෙන් විශේෂාංගයක් ආරක්ෂිතව සම්පූර්ණ කිරීමේ ඇල්ගොරිතමය ලියන්න.

```
1. git checkout -b feature      (create + switch)
2. edit the files               (code the feature)
3. git add .                   (stage)
4. git commit -m "message"     (snapshot)
5. git push                    (send to GitHub)
6. open Pull Request → review → merge into main
7. git branch -d feature       (clean up)
```

Used by every software team, from startups to large enterprises. Main stays clean throughout.

Q3. Write pseudocode that traverses a list using a manual offset (simulating $*(ptr+i)$) and prints each element in REVERSE.

අතින් offset එකක් භාවිතයෙන් $*(ptr+i)$ අනුකරණය ලැයිස්තුවක් ප්‍රතිලෝමව මුද්‍රණය කරන ව්‍යාජ කේතය ලියන්න.

```
SET offset ← length(scores) - 1
WHILE offset >= 0:
  OUTPUT scores[offset]      // dereference at offset
  offset ← offset - 1        // move pointer backwards
```

scores[offset] plays the role of $*(ptr + offset)$; decrementing the offset walks backwards through memory.

18. Flowchart Questions (2) ගැලීම් සටහන් ප්‍රශ්න (2)

Q1. Draw a flowchart for the nested-loop traversal of a 2D array (print every value). Describe each block.

ද්විමාන අරාවක කැඳලී ලූප ගමන සඳහා ගැලීම් සටහනක් අඳින්න.

Model description (draw with standard symbols — oval Start/End, diamond decisions, rectangles process):

```
(Start)
|
[r = 0]
|
<r < rows?> --No--> (End)
|Yes
[c = 0]
|
<c < cols?> --No--> [print newline; r = r + 1] --> back to <r < rows?>
|Yes
[print matrix[r][c]]
|
[c = c + 1] --> back to <c < cols?>
```

Outer decision controls rows; inner decision controls columns; the inner loop must finish before the row counter advances.

Q2. Draw a flowchart for resolving a merge conflict (from 'git merge' to the final commit).

ඒකාබද්ධ ගැටුමක් විසඳීම සඳහා ගැලීම් සටහනක් අඳින්න.

```
(Start: git merge branch)
|
<Conflict?> --No--> [merge completes (fast-forward / 3-way)] --> (End)
|Yes
[git status - shows 'both modified']
|
[Open file; locate <<<<<< ===== >>>>>> markers]
|
[Choose: yours / theirs / combine]
|
[Remove ALL markers; save]
|
<Markers remain?> --Yes--> back to [Remove ALL markers]
|No
[git add .] -> [git commit] -> (End)
```

The decision 'Markers remain?' captures the exam rule: no <<<<<<, =====, or >>>>>> may survive resolution.

19. Short Programming Questions (5) කෙටි ක්‍රමලේඛන ප්‍රශ්න (5)

Q1. Create the 3x3 matrix `[[2,4,6],[1,3,5],[7,8,9]]`, print it as a grid, and print the sum of all elements.

matrix එක ජාලයක් ලෙස මුද්‍රණය කර සියලු මූලද්‍රව්‍යවල එකතුව මුද්‍රණය කරන්න.

```
matrix = [[2,4,6],[1,3,5],[7,8,9]]
total = 0
for row in matrix:
    for v in row:
        print(v, end=" ")
        total += v
    print()
print("Sum:", total)      # Sum: 45
```

Q2. Find and print the largest value in the matrix above together with its `[row][col]` position.

විශාලතම අගය සහ පිහිටීම මුද්‍රණය කරන්න.

```
max_val, max_r, max_c = matrix[0][0], 0, 0
for r in range(len(matrix)):
    for c in range(len(matrix[r])):
        if matrix[r][c] > max_val:
            max_val, max_r, max_c = matrix[r][c], r, c
print(f"Largest {max_val} at [{max_r}][{max_c}]")  # 9 at [2][2]
```

Q3. Define a Student dataclass (name, age, gpa), create 3 students, and print each on one neatly formatted line.

Student dataclass එකක් අර්ථ දැක්වා, සිසුන් 3ක් සාදා, පිළිවෙලට මුද්‍රණය කරන්න.

```
from dataclasses import dataclass
@dataclass
class Student:
    name: str
    age: int
    gpa: float
students = [Student("Amal",25,3.8), Student("Nimal",27,3.5),
            Student("Kamala",24,3.9)]
for s in students:
    print(f"{s.name:10} | Age: {s.age} | GPA: {s.gpa}")
```

Q4. Traverse `scores = [85, 92, 78, 95, 60]` using a manual offset (no 'for x in scores') and print the maximum.

අනිත් offset භාවිතයෙන් උපරිමය සොයන්න.

```
scores = [85, 92, 78, 95, 60]
max_val = scores[0]
offset = 1
while offset < len(scores):
    if scores[offset] > max_val:
        max_val = scores[offset]
    offset += 1
print(max_val)      # 95
```

Q5. Demonstrate reference behaviour: show that a list reference shares changes but an int reference does not, verifying with `id()`.

ලැයිස්තු යොමුවක් වෙනස්කම් බෙදාගන්නා නමුත් int යොමුවක් නොබෙදන බව `id()` මගින් තහවුරු කර පෙන්වන්න.

```
a = [10]
ptr = a
ptr[0] = 50
print(a, id(a) == id(ptr))  # [50] True - shared object
b = 10
ref = b
ref = 50
print(b, id(b) == id(ref))  # 10 False - new object
```

20. Substantial Practical Problem — Student Record & Marks Analyzer

විශාල ප්‍රායෝගික ගැටලුව — ශිෂ්‍ය වාර්තා සහ ලකුණු විශ්ලේෂකය

The task (25 marks — model FGS-SUSL essay question) කාර්යය (ලකුණු 25)

A lecturer keeps the marks of 4 students across 3 subjects. Write ONE complete Python program that: කථිකාචාර්යවරයෙක් සිසුන් 4කගේ විෂය 3ක ලකුණු තබාගනී. පහත සියල්ල කරන එක් සම්පූර්ණ Python වැඩසටහනක් ලියන්න:

- (a) Defines a Student dataclass with fields name: str and marks: list — 4 marks
(අ) Student dataclass එක අර්ථ දක්වයි — ලකුණු 4
- (b) Builds a list of 4 Student objects (an array of structs; marks form the 4×3 2D structure) — 4 marks
(ආ) Student වස්තු 4ක ලැයිස්තුවක් සාදයි — ලකුණු 4
- (c) Prints a formatted marks table by traversing every student and every mark with nested loops — 5 marks
(ඇ) කැදලි දූප මගින් ලකුණු වගුව මුද්‍රණය කරයි — ලකුණු 5
- (d) Computes each student's average and finds the top student (loop comparison, no max()) — 5 marks
(ඈ) සාමාන්‍යය ගණනය කර හොඳම ශිෂ්‍යයා සොයයි — ලකුණු 5
- (e) Adds 5 bonus marks to subject 0 of the top student THROUGH A REFERENCE variable and proves with id() that the original object changed — 4 marks
(ඉ) යොමු විචල්‍යයක් හරහා ප්‍රසාද ලකුණු 5ක් එක් කර id() මගින් ඔප්පු කරයි — ලකුණු 4
- (f) States (as comments) the two Git commands to branch and merge this work safely — 3 marks
(ඊ) Git විධාන දෙක comments ලෙස සඳහන් කරයි — ලකුණු 3

Model solution ආදර්ශ විසඳුම

```
# (f) Safe workflow: git checkout -b feature-analyzer
# ...work, add, commit... then: git checkout main; git merge feature-analyzer
from dataclasses import dataclass

# (a) Structure – mixed types under one name
@dataclass
class Student:
    name: str
    marks: list          # 3 subject marks

# (b) Array of structs – each row is one record
students = [
    Student("Amal", [78, 85, 62]),
    Student("Nimal", [91, 74, 88]),
    Student("Kamala", [84, 95, 90]),
    Student("Saman", [55, 67, 73]),
]

# (c) Nested traversal – students (outer) x marks (inner)
print(f"{'Name':10} | S1 S2 S3")
for s in students:
    print(f"{s.name:10} |", end=" ")
    for m in s.marks:
        print(f"{m:3}", end=" ")
    print()

# (d) Average + top student via loop comparison
top = students[0]
top_avg = 0
for s in students:
    total = 0
    for m in s.marks:
        total += m
    avg = total / len(s.marks)
    print(f"{s.name:10} average = {avg:.1f}")
    if avg > top_avg:
        top, top_avg = s, avg
print(f"Top student: {top.name} ({top_avg:.1f})")
```



```
# (e) Modify through a reference – pointer behaviour
ref = top                                # like  sptr = s1 /  C: *sptr
ref.marks[0] += 5                        # bonus through the reference
print(top.marks[0])                     # changed – same object!
print(id(ref) == id(top))                # True – one object, two names
```

Expected output (with these marks): the table, four averages, 'Top student: Kamala (89.7)', then Kamala's S1 becomes 89 and the id() check prints True.

අපේක්ෂිත ප්‍රතිදානය: වගුව, සාමාන්‍ය හතර, 'Top student: Kamala (89.7)', ඉන්පසු Kamala ගේ S1 ලකුණ 89 වී id() පරීක්ෂාව True මුද්‍රණය කරයි.

Marking notes: correct dataclass (a); list of objects (b); TWO nested loops with end=" " and print() (c); accumulation + comparison without max() (d); reference sharing proven with id() (e); checkout -b and merge named (f).